

Hasta Memnuniyet Anketi - Dağıtım (Deployment) Rehberi

Hedef Altyapı: Neon (PostgreSQL) + Render (uygulama sunucusu) + Özel domain

Mimari Not: Bu proje ASP.NET Core Razor Pages monolitik yapıdadır — backend ve frontend tek bir uygulama olarak çalışır. Vercel gibi yalnızca statik/Node.js destekleyen platformlar .NET çalıştıramaz. Bu nedenle **tüm uygulama Render üzerinden** yayınlanır.

Genel Bakış (Sıralama)

1. Neon'da PostgreSQL veritabanı oluştur
2. Projeyi SQL Server → PostgreSQL'e geçir
3. Migration'ları yeniden oluştur
4. Dockerfile hazırla
5. Render'da Web Service oluştur ve deploy et
6. Veritabanı migration'larını çalıştır (Neon)
7. Özel domain'i Render'a bağla
8. İlk giriş ve doğrulama

ADIM 1 - Neon'da PostgreSQL Veritabanı Oluşturma

1. <https://console.neon.tech> adresine gidin ve hesabınıza giriş yapın (yoksa ücretsiz oluşturun).
2. **"New Project"** butonuna tıklayın.
3. Ayarlar:
 - **Project name:** HastaMemnuniyet (veya istediğiniz bir ad)
 - **Region:** Render sunucunuza en yakın bölge (ör. `eu-central-1` Avrupa için)
 - **PostgreSQL version:** 16 (varsayılan)
4. **"Create Project"** butonuna tıklayın.
5. Oluşturulan **Connection string**'i kopyalayın. Şu formatta olacaktır:

```
postgresql://kullanici:sifre@ep-xxxxx.eu-central-1.aws.neon.tech/neondb?sslmode=require
```

⚠ Bu bağlantı dizesini güvenli bir yere kaydedin. Render'da environment variable olarak kullanacaksınız.

ADIM 2 - Projeyi SQL Server → PostgreSQL'e Geçirme

2a) NuGet Paketini Değiştirin

`src/HastaMemnuniyet.Infrastructure/HastaMemnuniyet.Infrastructure.csproj` dosyasında:

ESKİ:

```
<PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer" Version="8.*" />
```

YENİ:

```
<PackageReference Include="Npgsql.EntityFrameworkCore.PostgreSQL" Version="8.*" />
```

2b) DependencyInjection.cs’de Provider’ı Değiştirin

src/HastaMemnuniyet.Infrastructure/DependencyInjection.cs dosyasında:

ESKİ (satır 34):

```
options.UseSqlServer(baglantiDizesi, sql =>
    sql.MigrationsAssembly(typeof(AppDbContext).Assembly.FullName));
```

YENİ:

```
options.UseNpgsql(baglantiDizesi, npgsql =>
    npgsql.MigrationsAssembly(typeof(AppDbContext).Assembly.FullName));
```

2c) appsettings.json Bağlantı Dizesini Güncelleyin

src/HastaMemnuniyet.Web/appsettings.json dosyasında (yerel geliştirme için):

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Host=localhost;Port=5432;Database=HastaMemnuniyetDb;Username=postgres;Password=postgres"
  }
}
```

Not: Üretimde (Render) bu değer environment variable ile override edilecek, buradaki değer yalnızca yerel geliştirme içindir.

2d) DateTime → DateTime.UtcNow Kontrolü

PostgreSQL, `DateTime` tipini `timestamp with time zone` olarak bekler. Kodda zaten `DateTime.UtcNow` kullanıyoruz — sorun olmamalı. Ancak `AppDbContext.OnModelCreating` içine şu satırı eklemeniz **şiddetle önerilir**:

```
// OnModelCreating metodunun başına ekleyin:
AppContext.SetSwitch("Npgsql.EnableLegacyTimestampBehavior", true);
```

Alternatif olarak `Program.cs` 'in en üstüne de ekleyebilirsiniz:

```
AppContext.SetSwitch("Npgsql.EnableLegacyTimestampBehavior", true);
var builder = WebApplication.CreateBuilder(args);
// ... geri kalan kod
```

Bu satır, PostgreSQL'in `DateTime` türlerini SQL Server benzeri (timezone-unaware) biçimde işlemesini sağlar.

ADIM 3 - Migration'ları Yeniden Oluşturma

Provider değiştiği için mevcut migration'lar SQL Server'a özeldir; temizlenip yeniden oluşturulmalıdır.

```
# 1. Eski migration dosyalarını silin
rm -rf src/HastaMemnuniyet.Infrastructure/Data/Migrations/

# 2. Projeyi derleyin (hata olmadığından emin olun)
dotnet build HastaMemnuniyet.sln

# 3. Yeni (PostgreSQL uyumlu) ilk migration'ı oluşturun
dotnet ef migrations add IlkOlusturmaPostgres \
  --project src/HastaMemnuniyet.Infrastructure \
  --startup-project src/HastaMemnuniyet.Web
```

Migration'lar yalnızca SQL dosyaları üretir, veritabanına henüz dokunmaz.

ADIM 4 - Dockerfile Hazırlama

Proje kök dizinine (`/HastaMemnuniyet/`) aşağıdaki `Dockerfile` 'ı oluşturun:

```
# ---- Derleme aşaması ----
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src

# Önce csproj dosyalarını kopyala ve restore et (katman cache'lemesi)
COPY src/HastaMemnuniyet.Domain/HastaMemnuniyet.Domain.csproj src/HastaMemnuniyet.Domain/
COPY src/HastaMemnuniyet.Application/HastaMemnuniyet.Application.csproj src/HastaMemnuniyet.Application/
COPY src/HastaMemnuniyet.Infrastructure/HastaMemnuniyet.Infrastructure.csproj src/HastaMemnuniyet.Infrastructure/
COPY src/HastaMemnuniyet.Web/HastaMemnuniyet.Web.csproj src/HastaMemnuniyet.Web/
RUN dotnet restore src/HastaMemnuniyet.Web/HastaMemnuniyet.Web.csproj

# Tüm kaynak kodu kopyala ve yayınlama
COPY . .
RUN dotnet publish src/HastaMemnuniyet.Web/HastaMemnuniyet.Web.csproj \
    -c Release -o /app/publish --no-restore

# ---- Çalıştırma aşaması ----
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS runtime
WORKDIR /app

# EF Core migration aracı için SDK katmanından kopyala
COPY --from=build /src/src/HastaMemnuniyet.Infrastructure/Data/Migrations/ /migrations/

COPY --from=build /app/publish .

# Render $PORT environment variable kullanır
ENV ASPNETCORE_URLS=http://+:${PORT:-10000}
ENV ASPNETCORE_ENVIRONMENT=Production

EXPOSE 10000

ENTRYPOINT ["dotnet", "HastaMemnuniyet.Web.dll"]
```

Ayrıca proje kök dizinine `.dockerignore` dosyası ekleyin:

```
*/bin
*/obj
*/node_modules
*/.git
*/Dockerfile*
*/.dockerignore
tests/
docs/
README.md
```

ADIM 5 - Render'da Web Service Oluşturma

5a) GitHub/GitLab'a Push Edin

Önce projeyi bir Git deposuna push etmeniz gerekir:

```
# GitHub'da yeni bir repo oluřturun, sonra:
cd HastaMemnuniyet
git remote add origin https://github.com/KULLANICI_ADINIZ/HastaMemnuniyet.git
git push -u origin main
```

5b) Render Dashboard'da Yeni Servis Oluřturun

1. <https://dashboard.render.com> adresine gidin.
2. “New +” → “Web Service” seřin.
3. “Build and deploy from a Git repository” seřin.
4. GitHub/GitLab hesabınızı baęlayın ve **HastaMemnuniyet** reposunu seřin.
5. Ayarları doldurun:

Alan	Deęer
Name	hasta-memnuniyet (veya istedięiniz ad)
Region	Neon veritabanınıza en yakın blge
Branch	main
Runtime	Docker
Dockerfile Path	./Dockerfile
Instance Type	Starter (\$7/ay) veya Free (sınırlı)

5c) Environment Variables (Ortam Deęiřkenleri) Ekleyin

Render servis ayarlarında “Environment” sekmesine gidin ve řu deęiřkenleri ekleyin:

Deęiřken	Deęer
ConnectionStrings__DefaultConnection	Host=ep-xxxxx.eu-cent- ral-1.aws.neon.tech;Port=5432;Database=neon db;Username=kullanici;Password=sifre;SSL Mode=Require;Trust Server Certificate=true
ASPNETCORE_ENVIRONMENT	Production
SeedVerisiniYukle	true (ilk kurulumda — seed sonrası false yapın)

⚠ **ÖNEMLİ:** Neon bağlantı dizesini `.NET` formatında yazmanız gerekir (Neon'un verdiği `postgres://...` formatını dönüştürün):

Neon verir:

```
postgresql://kullanici:sifre@ep-xxxxx.eu-central-1.aws.neon.tech/neondb?sslmode=require
```

Render'a gireceğiniz (.NET formatı):

```
Host=ep-xxxxx.eu-cent-  
ral-1.aws.neon.tech;Port=5432;Database=neondb;Username=kullanici;Password=sifre;SSL  
Mode=Require;Trust Server Certificate=true
```

5d) Deploy Edin

“**Create Web Service**” butonuna tıklayın. Render otomatik olarak:

1. Dockerfile'ı kullanarak imaj oluşturur
2. Container'ı başlatır
3. Size bir URL verir (ör. <https://hasta-memnuniyet.onrender.com>)

İlk deploy 3-5 dakika sürebilir. **Logs** sekmesinden ilerlemeyi takip edin.

ADIM 6 - Veritabanı Migration'larını Çalıştırma

Program.cs'e otomatik migration uygulama özelliği ekleyin. `SeedData` çağrısından **hemen önce** şu kodu ekleyin:

```
// Üretimde veritabanı şemasını otomatik güncelle  
using (var scope = app.Services.CreateScope())  
{  
    var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();  
    await db.Database.MigrateAsync();  
}
```

Bu sayede uygulama her başladığında bekleyen migration'ları otomatik uygular.

Alternatif olarak, migration'ları yerel makinenizden doğrudan Neon'a çalıştırabilirsiniz:

```
```bash
```

## Neon bağlantı dizesini geçici olarak appsettings.json'a koyun, sonra:

```
dotnet ef database update \
-project src/HastaMemnuniyet.Infrastructure \
-startup-project src/HastaMemnuniyet.Web
```
```

ADIM 7 - Özel Domain'i Render'a Bağlama

7a) Render'da Domain Ekleme

1. Render Dashboard → servisinizi seçin → **"Settings"** → **"Custom Domains"**
2. **"Add Custom Domain"** butonuna tıklayın
3. Domain adınızı girin (ör. `anket.sizindomain.com` veya `sizindomain.com`)
4. Render size DNS ayarlarını gösterecek

7b) DNS Kayıtlarını Ayarlama

Domain sağlayıcınızın (GoDaddy, Namecheap, Cloudflare vb.) DNS yönetim paneline gidin:

Alt alan adı kullanıyorsanız (ör. `anket.sizindomain.com`):

| Tip | Ad | Değer |
|-------|-------|--|
| CNAME | anket | Render'ın verdiği hedef (ör. <code>xxx.onrender.com</code>) |

Kök domain kullanıyorsanız (ör. `sizindomain.com`):

| Tip | Ad | Değer |
|-----|----|-----------------------------|
| A | @ | Render'ın verdiği IP adresi |

DNS yayılması 5 dakika ile 48 saat arasında sürebilir (genellikle 5-30 dakika).

7c) SSL Sertifikası

Render, Let's Encrypt kullanarak **otomatik ücretsiz SSL** sertifikası oluşturur. DNS doğrulandıktan sonra `https://sizindomain.com` çalışır hale gelir.

ADIM 8 - İlk Giriş ve Doğrulama

1. Tarayıcıda sitenizi açın: `https://sizindomain.com` (veya Render URL'si)
2. Giriş sayfası gelmelidir
3. Seed data yüklendiğinde varsayılan yönetici hesabıyla giriş yapın:
 - **E-posta:** `admin@hastamemnuniyet.com` (SeedData'da tanımlı)
 - **Şifre:** `Admin123!` (SeedData'da tanımlı)

⚠ İLK İŞ: Giriş yaptıktan sonra yönetici şifresini hemen değiştirin!

1. Doğrulama kontrol listesi:
 - [] Dashboard açılıyor, NPS kartı görünüyor
 - [] Raporlar menüsü açılıyor (Anahtar Kelime, Dönemsel, Doktor Karnesi)
 - [] Yeni anket oluşturulabiliyor
 - [] Davet gönderme ekranı çalışıyor
 - [] Excel dışı aktarım çalışıyor

Sorun Giderme

“Connection refused” veya veritabanı bağlantı hatası

- Neon bağlantı dizesinin `.NET` formatında olduğundan emin olun
- `SSL Mode=Require;Trust Server Certificate=true` eklediğinizden emin olun
- Render environment variable adının **tam olarak** `ConnectionStrings__DefaultConnection` olduğunu doğrulayın (çift alt çizgi `__`)

Migration hatası

- Render loglarında hata varsa, migration’ları yerel makineden Neon’a doğrudan çalıştırın
- `dotnet ef migrations add` komutunu **PostgreSQL provider yüklüyken** çalıştırdığınızdan emin olun

Türkçe karakter sorunu

- PostgreSQL varsayılan olarak UTF-8 kullanır, Türkçe karakterler sorunsuz çalışır
- Neon veritabanı oluşturulurken encoding zaten `UTF8` ‘dir

Render “Deploy failed”

- **Logs** sekmesini kontrol edin
- Dockerfile’da `dotnet restore` başarısız oluyorsa, NuGet paket adını doğrulayın
- `dotnet publish` hatası varsa, önce yerel makinede `dotnet publish -c Release` çalıştırdığınızdan emin olun

Maliyet Özeti (Yaklaşık)

| Servis | Plan | Aylık Maliyet |
|---------------|---------------------------------|----------------|
| Neon | Free (0.5 GB, 1 proje) | \$0 |
| Render | Starter (512 MB RAM) | \$7 |
| Domain | (zaten mevcut) | - |
| SSL | Render otomatik (Let’s Encrypt) | \$0 |
| Toplam | | ~\$7/ay |

Neon free planı küçük-orta ölçekli projeler için yeterlidir. Render free planı da mevcuttur ancak 15 dakika inaktiviteden sonra uyku moduna girer (ilk istek yavaş olur).

Özet Akış Şeması

